

UNIT 2, BUILDING INTELLIGENT AGENT WORKFLOWS

LECTURE 4

Workflow Chaining and Event-Driven Systems

One tool call was the atom. Now we build the molecule:
steps chained into a pipeline, and systems that react to
things happening rather than waiting to be asked.

SECTION 00

From one call to many

You can call a service safely. Now chain several into real work.

RECALL

Where we are

LECTURE 1

Chose the shape

Workflow, router, or agent. Who decides the next step.

LECTURE 2

Survived tool failure

`call_tool` turns any broken tool into a readable observation.

LECTURE 3

Reached real services

The wrapper hides a messy API behind a clean tool.

THE SHIFT

Every tool call so far has stood alone: ask, answer, done. Real automation is a chain. A ticket comes in, and four things have to happen to it, in order, each depending on the last. Today we build that chain, and handle the moment a link in it breaks.

Today

- The pipeline: named steps in order, passing state from one to the next.
- The single most important property of a pipeline, and it is about failure.
- Why a broken step three must never run on a half-finished step two.
- The event-driven shape: work that starts because something happened.
- When to reach for a pipeline, and when to reach for events.

HOLD ON TO THIS

We stay with support ticket triage from Lecture 1, but the single routing decision grows up into a real intake pipeline: validate the ticket, classify it, enrich it with an SLA, and assign it. Four steps, in order, each needing the one before.

SECTION 01

The pipeline

Named steps, in an order you chose.

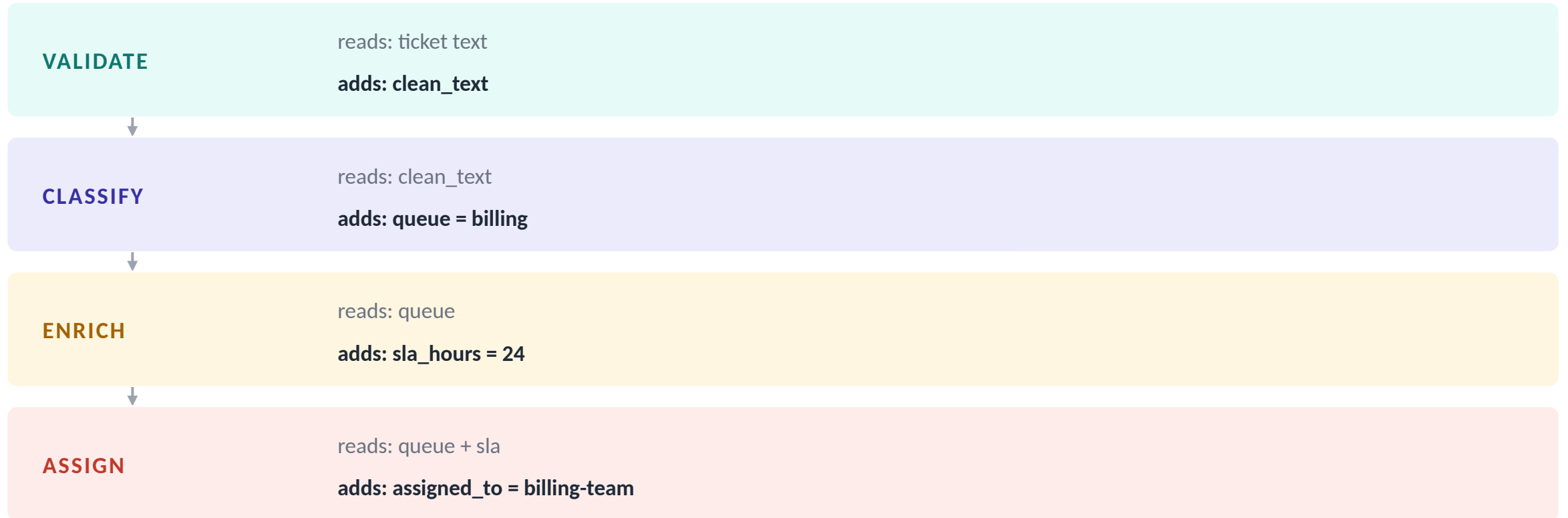
A pipeline is a workflow, grown up

```
pipeline = (  
    Pipeline("ticket-intake")  
    .step("validate", validate_ticket)  
    .step("classify", classify_ticket)  
    .step("enrich",    enrich_ticket)  
    .step("assign",    assign_ticket)  
)  
  
state, log = pipeline.run({"text": ticket})
```

READ IT TOP TO BOTTOM

The order is right there in the code, written by you, in advance. That is exactly the definition of a workflow from Lecture 1: you decided the steps, the model did not. The pipeline just runs them and threads a shared state from each step into the next. Nothing here reasons. That is a feature, not a limitation.

State flows down the chain



Each step reads what earlier steps left behind and adds its own. By the end, the state carries the whole story of the ticket.

SECTION 02

When a link breaks

The most important slide in the lecture is about failure.

The rule that makes pipelines safe

When a step fails, stop. Never run the next step on a half-finished state.

Step three assumes step two succeeded. If classify failed and left no queue, then enrich has nothing to enrich, and assign has nothing to assign. Running them anyway does not produce nothing. It produces a confident, wrong result, which is far worse.

FAIL FAST, AND KEEP WHAT WORKED

The pipeline stops at the failed step and returns the state as it stood. The work that already succeeded is preserved, the broken step is named in the log, and nothing downstream runs on bad data. A clean stop with a clear reason beats a full run with a wrong answer, every time.

Watch it stop cleanly

```
>>> state, log = pipeline.run({"text": ""})    # empty ticket

[FAIL] validate: ticket has no text
-> stopped at validate

# classify never ran. enrich never ran. assign never ran.
# no queue was invented. no team was wrongly assigned.
```

THE LOG IS OBSERVABILITY, ARRIVING EARLY

Notice the run produced a log: which steps ran, which passed, where it stopped, and why. That log is the difference between "it did not work" and "it stopped at validate because the ticket had no text". You will build much more of this in Unit 5, but the habit starts here: every run should be able to explain itself.

This is the workflow decision, with more at stake

WHY NOT AN AGENT HERE

The steps are known

Validate, classify, enrich, assign. Same four, same order, every ticket. You can write them in advance, so from Lecture 1, you should. An agent choosing these four steps freshly each time would be pure waste.

WHAT CHANGED SINCE L1

Now failure compounds

With one call, a failure was one failure. In a chain, a failure partway through can corrupt everything after it. The cost of choosing the wrong shape got higher, which makes choosing the simple, testable workflow matter more, not less.

HOLD ON TO THIS

The three-question test from Lecture 1 still decides it. Can you write the steps in advance? For intake, yes. So it is a workflow, and the pipeline is how you build one cleanly.

SECTION 03

Event-driven systems

Work that starts because something happened.

Two directions of control

PIPELINE

A line you walk down

You start at the top and move through the steps yourself, in order, one after another. You are in control the whole way. Good when you know the sequence and you drive it.

EVENT BUS

A room you shout into

You announce that something happened, and whoever was listening for that reacts. You do not know or care who responds. Good when one thing happening should trigger many independent reactions.

A CONCRETE EXAMPLE

A ticket is created. In a pipeline, you call validate, then classify, then assign. In an event system, you shout "ticket.created" into the room, and independently a logger records it, a notifier emails the team, and a metrics counter ticks up. None of those three know about each other. Adding a fourth reaction later changes nothing that already exists.

The event bus, in code

```
bus = EventBus()

bus.on("ticket.created", log_it)
bus.on("ticket.created", notify_team)
bus.on("ticket.created", bump_metric)

bus.emit(Event("ticket.created", {"id": 7}))
# all three react. none knows about the others.
```

ONE BROKEN LISTENER MUST NOT SILENCE THE REST

If `notify_team` throws because the email server is down, `log_it` and `bump_metric` must still run. Our bus catches a failing handler and moves on, because in an event system the whole point is that reactions are independent. One broken reaction is not allowed to cancel the others.

Which one, when

USE A PIPELINE

Ordered steps, each needing the last, one clear outcome. Intake, checkout, a data transform. You know the sequence and you drive it.

USE EVENTS

One thing happens, several independent things should react, and you want to add reactions later without touching the trigger. Notifications, logging, metrics, side effects.

OFTEN, BOTH

A pipeline finishes and emits an event. The intake pipeline assigns a ticket, then shouts "ticket.assigned", and independent handlers react. The shapes compose.

HOLD ON TO THIS

You do not choose one forever. A mature system is pipelines for the ordered work and events for the reactions around it. Knowing which question each shape answers is what lets you combine them well.

Hold on to this

A pipeline is a line you walk. An event bus is a room you shout into.

State flows down the chain

Each step reads what came before and adds its own.

Stop on the first failure

A wrong answer is worse than an honest stop. Keep what worked.

Every run explains itself

The log says what ran and where it stopped. Observability starts here.

Pipelines order, events react

And they compose: a pipeline can end by emitting an event.

Before next session

DO

Add a fifth step

Insert a "deduplicate" step into the intake pipeline that stops the run if the same ticket text was already seen. Notice you only touched one place, not the whole chain.

DO

Break a middle step

Make classify fail on purpose. Confirm the pipeline stops there, keeps the validate result, and never assigns a ticket to a nonexistent queue. Read the log.

THINK

Pipeline or events

For your capstone, name one part that is genuinely ordered steps, and one part that is really independent reactions to an event. Do not force everything into one shape.

ON THE FIRST ONE

Adding a step touches one line and nothing else. That is the whole payoff of building the pipeline cleanly. If adding a step meant rewriting the run logic, the design would be wrong. Ease of change is how you judge a structure.

Next session

Unit 2, Lecture 5

Context-Aware Agents and Workflow Testing

The last lecture of Unit 2. We give agents memory that carries across turns, so a conversation can build on what came before, and we take the first serious look at testing these systems, which is the run log from today grown into a real discipline. Then Unit 2 is done, and you can build, chain, and begin to trust an agent workflow.

COME WITH

Your fifth pipeline step added, your broken-step experiment run, and one honest pipeline-versus-events split from your capstone.